

EX No:1b	EDA DATA SCIENCE FRAMEWORKS (SEMMA, KDD AND CRISP-DM)

Aim

To analyze the Restaurant Sales dataset using Exploratory Data Analysis (EDA), data preprocessing, statistical analysis and visualization, and to demonstrate the SEMMA, KDD and CRISP-DM data science frameworks for restaurant sales analysis and sales prediction.

Procedure

1. Load the Restaurant Sales dataset using Pandas.
2. Import the required Python libraries such as Pandas, NumPy, Matplotlib, Seaborn and Scikit-learn.
3. Display the first few records of the dataset.
4. Display the dataset shape, information, data types and missing values.
5. Check and remove duplicate records if present.
6. Clean text columns and convert the Date column into datetime format.
7. Create the Sales field using Price $\times$ Quantity.
8. Perform SEMMA: Sample, Explore, Modify, Model and Assess.
9. Perform KDD: Selection, Preprocessing, Transformation, Data Mining and Interpretation/Evaluation.
10. Perform CRISP-DM: Business Understanding, Data Understanding, Data Preparation, Modeling, Evaluation and Deployment.
11. Visualize product-wise, city-wise, purchase-type and time-based sales.
12. Perform grouping, aggregation and correlation analysis.
13. Encode categorical variables for machine learning.
14. Split the dataset into training and testing data.
15. Train a Random Forest Regression model to predict Sales.
16. Evaluate the model using MAE, MSE, RMSE and R^2 score.

17. Display actual versus predicted sales and generate business insights.

Python Code

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns

from sklearn.model_selection import train_test_split
from sklearn.ensemble import RandomForestRegressor
from sklearn.metrics import mean_absolute_error, mean_squared_error, r2_score

# 1. Load dataset
df = pd.read_csv("9. Sales-Data-Analysis.csv")

# 2. Display data
print(df.head())

# 3. Basic information
print("Shape:", df.shape)
print(df.info())

# 4. Check missing values and duplicates
print(df.isnull().sum())
print("Duplicates:", df.duplicated().sum())

# 5. Clean column names and text data
df.columns = df.columns.str.strip()

for col in df.select_dtypes(include="object").columns:
    df[col] = df[col].str.strip()
    df[col] = df[col].str.replace(r"\s+", " ", regex=True)

# 6. Convert Date
df["Date"] = pd.to_datetime(df["Date"], dayfirst=True)
```

```
# 7. Create Sales
```

```
df["Sales"] = df["Price"] * df["Quantity"]
```

```
print(df.head())
```

SEMMA Framework

SEMMA stands for Sample, Explore, Modify, Model and Assess.

S — Sample

```
# Take a sample of the dataset
```

```
sample_df = df.sample(frac=0.8, random_state=42)
```

```
print("Original Shape:", df.shape)
```

```
print("Sample Shape:", sample_df.shape)
```

E — Explore

```
# Statistical summary
```

```
print(sample_df.describe())
```

```
# Product-wise sales
```

```
product_sales = sample_df.groupby("Product")["Sales"].sum()
```

```
print(product_sales)
```

```
# City-wise sales
```

```
city_sales = sample_df.groupby("City")["Sales"].sum()
```

```
print(city_sales)
```

```
# Daily sales trend
```

```
daily_sales = sample_df.groupby("Date")["Sales"].sum()
```

```
daily_sales.plot(kind="line", figsize=(10, 5))
```

```
plt.title("Daily Sales Trend")
```

```
plt.xlabel("Date")
```

```
plt.ylabel("Sales")
```

```
plt.show()
```

```
# Sales by product
```

```
product_sales.sort_values(ascending=False).plot(kind="bar", figsize=(8, 5))
```

```
plt.title("Sales by Product")
plt.xlabel("Product")
plt.ylabel("Sales")
plt.xticks(rotation=45)
plt.show()
```

M — Modify

```
# Create date features
df["Day"] = df["Date"].dt.day
df["Month"] = df["Date"].dt.month
df["DayOfWeek"] = df["Date"].dt.dayofweek

# Encode categorical columns
model_df = pd.get_dummies(
    df,
    columns=["Product", "Purchase Type", "Payment Method", "Manager", "City"],
    drop_first=True
)
```

SEMMA — Model

```
X = model_df.drop(columns=["Sales", "Date"])
y = model_df["Sales"]

X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42
)

model = RandomForestRegressor(
    n_estimators=100,
    random_state=42
)

model.fit(X_train, y_train)

y_pred = model.predict(X_test)

print("Model training completed.")
```

SEMMA — Assess

```
mae = mean_absolute_error(y_test, y_pred)
mse = mean_squared_error(y_test, y_pred)
rmse = np.sqrt(mse)
r2 = r2_score(y_test, y_pred)
```

```
print("MAE:", mae)
print("MSE:", mse)
print("RMSE:", rmse)
print("R2 Score:", r2)
```

```
plt.figure(figsize=(8, 5))
plt.scatter(y_test, y_pred)
plt.xlabel("Actual Sales")
plt.ylabel("Predicted Sales")
plt.title("Actual vs Predicted Sales")
plt.show()
```

KDD Framework

KDD stands for Knowledge Discovery in Databases. The stages used are Selection, Preprocessing, Transformation, Data Mining and Interpretation/Evaluation.

1. Selection

```
kdd_df = df[
    ["Date", "Product", "Price", "Quantity",
     "Purchase Type", "Payment Method",
     "Manager", "City", "Sales"]
].copy()
```

```
display(kdd_df.head())
```

2. Preprocessing

```
print("Missing Values:")
print(kdd_df.isnull().sum())
```

```
print("Duplicate Rows:", kdd_df.duplicated().sum())
```

```
kdd_df = kdd_df.drop_duplicates()
kdd_df = kdd_df.dropna()
```

```
print("After preprocessing:", kdd_df.shape)
```

3. Transformation

```
kdd_df["Day"] = kdd_df["Date"].dt.day
kdd_df["Month"] = kdd_df["Date"].dt.month
kdd_df["Year"] = kdd_df["Date"].dt.year
kdd_df["DayOfWeek"] = kdd_df["Date"].dt.dayofweek

monthly_sales = kdd_df.groupby("Month")["Sales"].sum()

monthly_sales.plot(kind="bar", figsize=(7, 5))
plt.title("Monthly Sales")
plt.xlabel("Month")
plt.ylabel("Sales")
plt.show()
```

KDD — 4. Data Mining

```
# Best-selling products
best_products = (
    kdd_df.groupby("Product")["Sales"]
    .sum()
    .sort_values(ascending=False)
)

print("Best-selling Products:")
print(best_products)

# Best-performing cities
best_city = (
    kdd_df.groupby("City")["Sales"]
    .sum()
    .sort_values(ascending=False)
)

print("Sales by City:")
print(best_city)
```

```
# Correlation analysis
numeric_columns = [
    "Price", "Quantity", "Sales",
    "Day", "Month", "DayOfWeek"
]

correlation = kdd_df[numeric_columns].corr()

plt.figure(figsize=(8, 6))
sns.heatmap(correlation, annot=True, cmap="coolwarm")
plt.title("Sales Data Correlation Heatmap")
plt.show()
```

KDD — 5. Interpretation / Evaluation

```
print("Highest Selling Product:", best_products.idxmax())
print("Highest Sales City:", best_city.idxmax())
print("Total Sales:", kdd_df["Sales"].sum())
print("Average Sales:", kdd_df["Sales"].mean())
```

CRISP-DM Framework

CRISP-DM stands for Cross-Industry Standard Process for Data Mining. The six stages are Business Understanding, Data Understanding, Data Preparation, Modeling, Evaluation and Deployment.

1. Business Understanding

```
print("""
Business Objective:
Analyze restaurant sales and predict sales using historical data.
```

Goals:

1. Understand restaurant sales patterns.
 2. Identify best-selling products.
 3. Analyze sales by city and purchase type.
 4. Predict future sales.
 5. Support inventory and business planning.
- ```
""")
```

### **2. Data Understanding**

```
print("Rows:", df.shape[0])
print("Columns:", df.shape[1])
```

```
df.info()
print(df.describe())
```

```
print("Missing Values:")
print(df.isnull().sum())
```

### **CRISP-DM — 3. Data Preparation**

```
crisp_df = df.copy()

Remove duplicates and missing records
crisp_df = crisp_df.drop_duplicates()
crisp_df = crisp_df.dropna()

Extract date features
crisp_df["Day"] = crisp_df["Date"].dt.day
crisp_df["Month"] = crisp_df["Date"].dt.month
crisp_df["Year"] = crisp_df["Date"].dt.year
crisp_df["DayOfWeek"] = crisp_df["Date"].dt.dayofweek

Encode categorical variables
crisp_df = pd.get_dummies(
 crisp_df,
 columns=[
 "Product", "Purchase Type",
 "Payment Method", "Manager", "City"
],
 drop_first=True
)

display(crisp_df.head())
```

### **CRISP-DM — 4. Modeling**

```
X = crisp_df.drop(columns=["Sales", "Date"])
y = crisp_df["Sales"]
```

```
X_train, X_test, y_train, y_test = train_test_split(
 X, y, test_size=0.2, random_state=42
)
```

```
rf_model = RandomForestRegressor(
 n_estimators=100,
 random_state=42
)
```

```
rf_model.fit(X_train, y_train)
```

```
predictions = rf_model.predict(X_test)
```

```
print("Model created successfully.")
```

### **CRISP-DM — 5. Evaluation**

```
mae = mean_absolute_error(y_test, predictions)
mse = mean_squared_error(y_test, predictions)
rmse = np.sqrt(mse)
r2 = r2_score(y_test, predictions)
```

```
print("MAE:", mae)
print("MSE:", mse)
print("RMSE:", rmse)
print("R2 Score:", r2)
```

```
plt.figure(figsize=(10, 5))
plt.plot(y_test.values, label="Actual Sales")
plt.plot(predictions, label="Predicted Sales")
plt.title("Actual vs Predicted Restaurant Sales")
plt.xlabel("Test Samples")
plt.ylabel("Sales")
plt.legend()
plt.show()
```

### **CRISP-DM — 6. Deployment / Business Insights**

```
total_sales = df["Sales"].sum()
```

```

top_product = (
 df.groupby("Product")["Sales"].sum().idxmax()
)

top_city = (
 df.groupby("City")["Sales"].sum().idxmax()
)

print("Total Sales:", total_sales)
print("Top Product:", top_product)
print("Top City:", top_city)

print(
 "Recommendation: Use historical sales patterns "
 "to improve inventory planning, product availability "
 "and future sales forecasting."
)

```

## OUTPUT

```

*** DATASET LOADED SUCCESSFULLY
=====
First 5 Records:
 Order ID Date Product Price Quantity Purchase Type \
0 10452 07-11-2022 Fries 3.49 573.07 Online
1 10453 07-11-2022 Beverages 2.95 745.76 Online
2 10454 07-11-2022 Sides & Other 4.99 200.40 In-store
3 10455 08-11-2022 Burgers 12.99 569.67 In-store
4 10456 08-11-2022 Chicken Sandwiches 9.95 201.01 In-store

 Payment Method Manager City
0 Gift Card Tom Jackson London
1 Gift Card Pablo Perez Madrid
2 Gift Card Joao Silva Lisbon
3 Credit Card Walter Muller Berlin
4 Credit Card Walter Muller Berlin

Dataset Shape:
(254, 9)

Column Names:
['Order ID', 'Date', 'Product', 'Price', 'Quantity', 'Purchase Type', 'Payment Method', 'Manager', 'City']

Dataset Information:
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 254 entries, 0 to 253
Data columns (total 9 columns):
Column Non-Null Count Dtype
--- -
0 Order ID 254 non-null int64
1 Date 254 non-null object
2 Product 254 non-null object
3 Price 254 non-null float64
4 Quantity 254 non-null float64
5 Purchase Type 254 non-null object
6 Payment Method 254 non-null object
7 Manager 254 non-null object
8 City 254 non-null object
dtypes: float64(2), int64(1), object(6)
memory usage: 18.0+ KB
None

```

```

Statistical Summary:
 Order ID Price Quantity
count 254.000000 254.000000 254.000000
mean 10584.133858 7.102323 460.611457
std 75.889181 4.341855 214.888699
min 10452.000000 2.950000 200.400000
25% 10520.250000 3.490000 201.010000
50% 10583.500000 4.990000 538.880000
75% 10649.750000 9.950000 677.440000
max 10713.000000 29.050000 754.430000

```

```

Missing Values:
Order ID 0
Date 0
Product 0
Price 0
Quantity 0
Purchase Type 0
Payment Method 0
Manager 0
City 0
dtype: int64

```

```

Duplicate Rows:
0

```

```

After Data Cleaning:
(254, 9)

```

```

Missing Values After Cleaning:
Order ID 0
Date 0
Product 0
Price 0
Quantity 0
Purchase Type 0
Payment Method 0
Manager 0
City 0
dtype: int64

```

```

Sales Column Created:
 Product Price Quantity Sales
0 Fries 3.49 573.07 2000.0143
1 Beverages 2.95 745.76 2199.9920
2 Sides & Other 4.99 200.40 999.9960
3 Burgers 12.99 569.67 7400.0133
4 Chicken Sandwiches 9.95 201.01 2000.0495

```

```

=====
SEMMA - SAMPLE
=====

```

```

Original Dataset Shape: (254, 10)

```

```

Sample Dataset Shape: (203, 10)

```

```

Sample Data:

```

```

 Order ID Date Product Price Quantity Purchase Type \
190 10650 2022-12-17 Burgers 12.99 585.07 Online
6 10459 2022-11-08 Sides & Other 4.99 200.40 In-store
79 10536 2022-11-24 Chicken Sandwiches 9.95 201.01 Drive-thru
205 10665 2022-12-20 Burgers 12.99 646.65 In-store
117 10577 2022-12-02 Fries 3.49 630.37 Online

```

```

 Payment Method Manager City Sales
190 Gift Card Joao Silva Lisbon 7600.0593
6 Credit Card Walter Muller Berlin 999.9960
79 Credit Card Pablo Perez Madrid 2000.0495
205 Cash Joao Silva Lisbon 8399.9835
117 Credit Card Remy Monet Paris 2199.9913

```

# SEMMA - EXPLORE

## Statistical Summary:

|       | Order ID     | Date                          | Price      | Quantity \ |
|-------|--------------|-------------------------------|------------|------------|
| count | 203.000000   | 203                           | 203.000000 | 203.000000 |
| mean  | 10581.817734 | 2022-12-02 23:31:37.536945920 | 7.220099   | 464.011576 |
| min   | 10452.000000 | 2022-11-07 00:00:00           | 2.950000   | 200.400000 |
| 25%   | 10514.500000 | 2022-11-19 12:00:00           | 3.490000   | 201.010000 |
| 50%   | 10583.000000 | 2022-12-03 00:00:00           | 4.990000   | 554.270000 |
| 75%   | 10644.500000 | 2022-12-15 12:00:00           | 9.950000   | 677.705000 |
| max   | 10712.000000 | 2022-12-29 00:00:00           | 29.050000  | 745.760000 |
| std   | 76.245119    | NaN                           | 4.470896   | 211.854352 |

## Sales

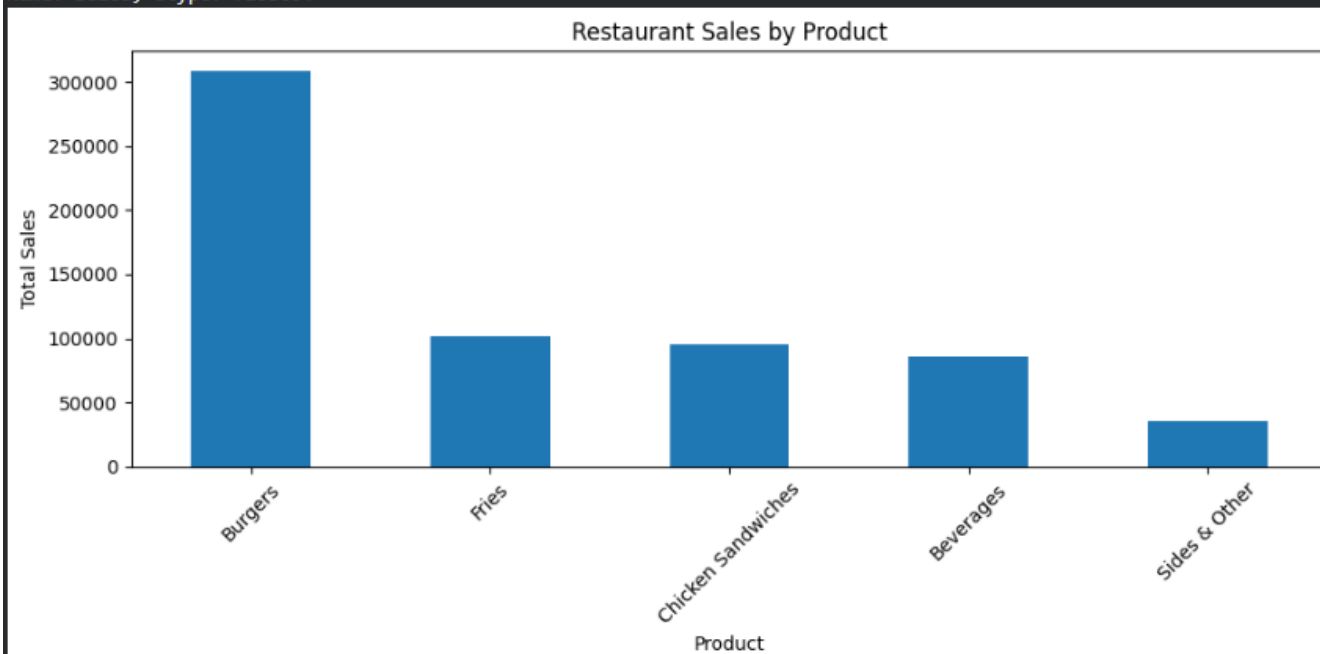
|       |              |
|-------|--------------|
| count | 203.000000   |
| mean  | 3084.312958  |
| min   | 999.996000   |
| 25%   | 2000.011500  |
| 50%   | 2000.049500  |
| 75%   | 2400.039500  |
| max   | 16074.435000 |
| std   | 2437.123698  |

## Sales by Product:

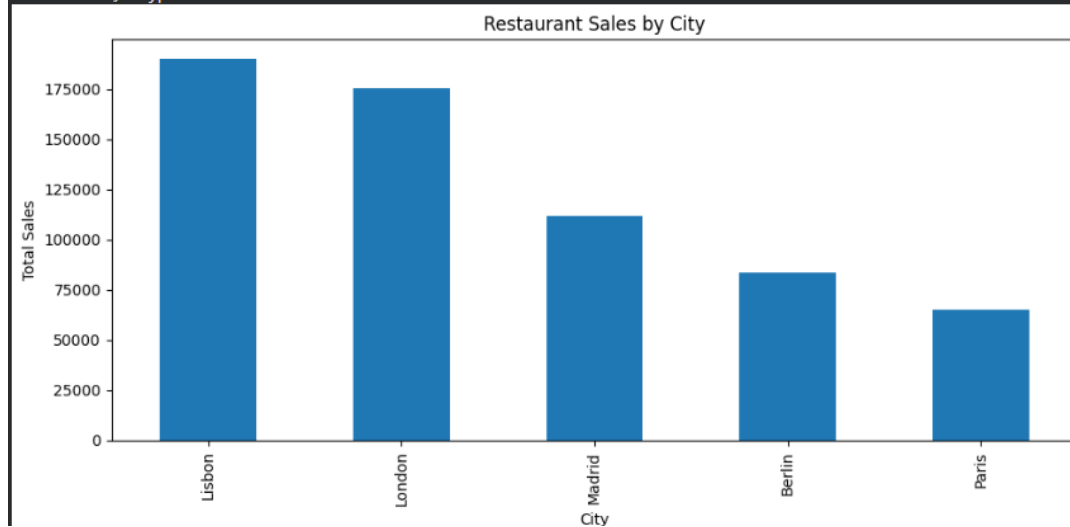
| Product            |             |
|--------------------|-------------|
| Burgers            | 308399.7468 |
| Fries              | 101474.3511 |
| Chicken Sandwiches | 95241.2845  |
| Beverages          | 86000.2880  |
| Sides & Other      | 34999.8600  |

Name: Sales, dtype: float64

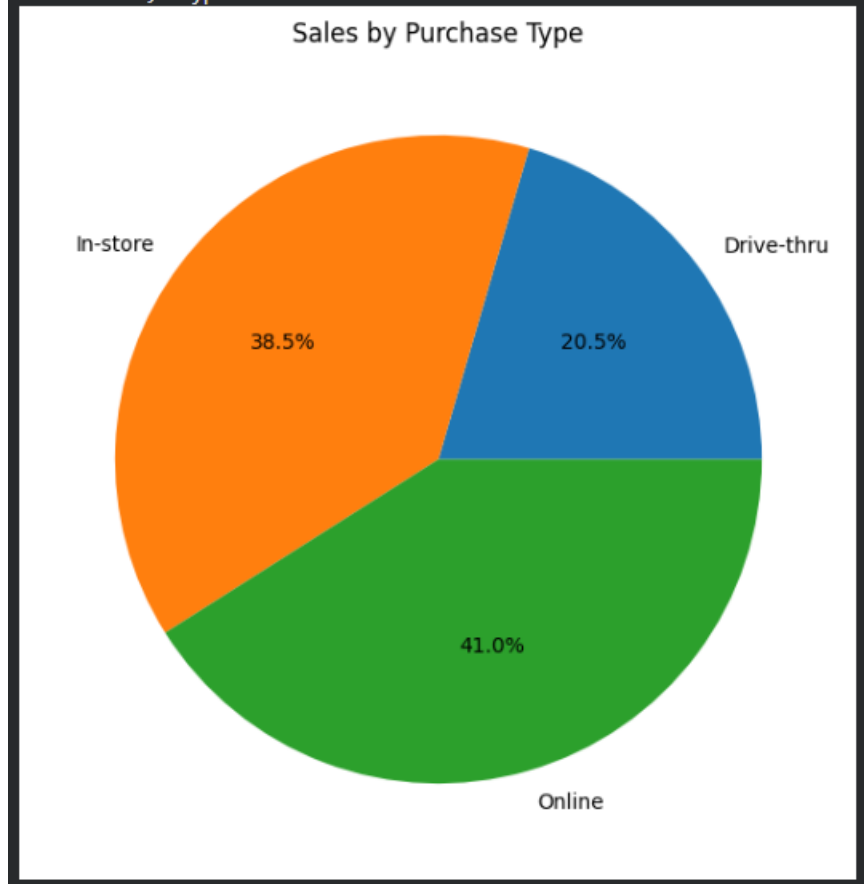
Name: Sales, dtype: float64



```
Sales by City:
City
Lisbon 190114.1717
London 175200.9630
Madrid 111800.2166
Berlin 83800.0576
Paris 65200.1215
Name: Sales, dtype: float64
```



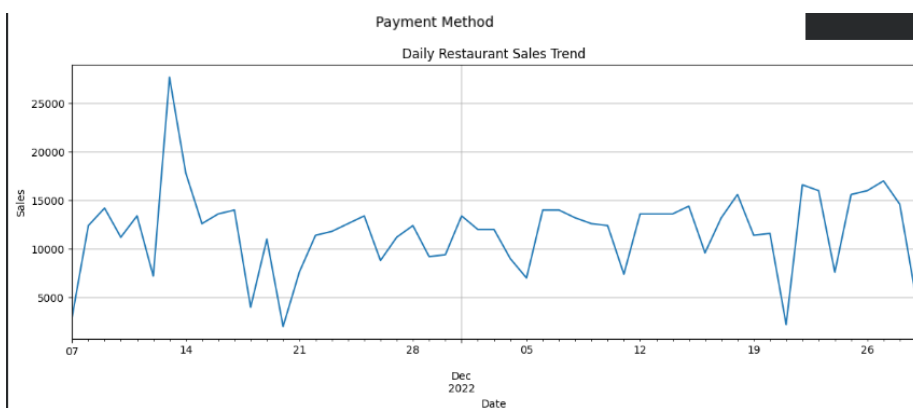
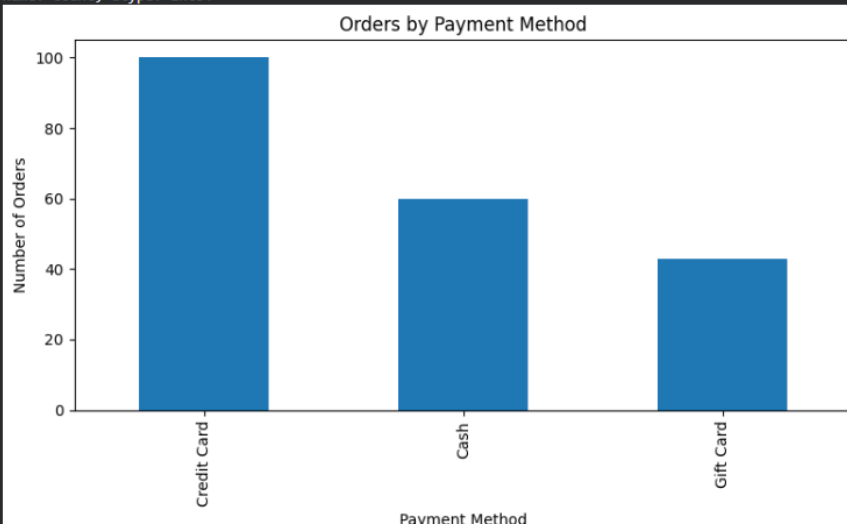
```
Sales by Purchase Type:
Purchase Type
Drive-thru 128200.2344
In-store 241114.1063
Online 256801.1897
Name: Sales, dtype: float64
```



```

Orders by Payment Method:
Payment Method
Credit Card 100
Cash 60
Gift Card 43
Name: count, dtype: int64

```



```

=====
SEMMA - MODIFY
=====

```

```

New Features Created:
 Date Day Month Year DayOfWeek
0 2022-11-07 7 11 2022 0
1 2022-11-07 7 11 2022 0
2 2022-11-07 7 11 2022 0
3 2022-11-08 8 11 2022 1
4 2022-11-08 8 11 2022 1

```

```

After One-Hot Encoding:
(254, 25)

```

```

=====
SEMMA - MODEL
=====

```

```

Training Data: (203, 23)
Testing Data: (51, 23)

```

```

Model Training Completed!

```

```

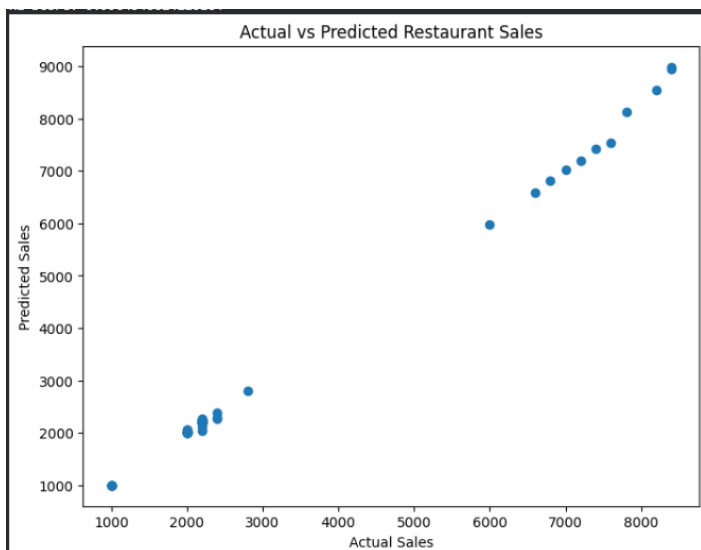
=====
SEMMA - ASSESS
=====

```

```

Mean Absolute Error (MAE): 57.77373168627352
Mean Squared Error (MSE): 18800.63379886508
Root Mean Squared Error (RMSE): 137.11540321519345
R2 Score: 0.9964548624210184

```



## KDD - SELECTION

Selected Columns:

```
Selected Columns:
['Date', 'Product', 'Price', 'Quantity', 'Purchase Type', 'Payment Method', 'Manager', 'City', 'Sales']
```

**Selected Data:**

|   | Date       | Product            | Price | Quantity | Purchase Type |
|---|------------|--------------------|-------|----------|---------------|
| 0 | 2022-11-07 | Fries              | 3.49  | 573.07   | Online        |
| 1 | 2022-11-07 | Beverages          | 2.95  | 745.76   | Online        |
| 2 | 2022-11-07 | Sides & Other      | 4.99  | 200.40   | In-store      |
| 3 | 2022-11-08 | Burgers            | 12.99 | 569.67   | In-store      |
| 4 | 2022-11-08 | Chicken Sandwiches | 9.95  | 201.01   | In-store      |

|   | Payment Method | Manager       | City   | Sales     |
|---|----------------|---------------|--------|-----------|
| 0 | Gift Card      | Tom Jackson   | London | 2000.0143 |
| 1 | Gift Card      | Pablo Perez   | Madrid | 2199.9920 |
| 2 | Gift Card      | Joao Silva    | Lisbon | 999.9960  |
| 3 | Credit Card    | Walter Muller | Berlin | 7400.0133 |
| 4 | Credit Card    | Walter Muller | Berlin | 2000.0495 |

## KDD - PREPROCESSING

Missing Values:

```
Date 0
Product 0
Price 0
Quantity 0
Purchase Type 0
Payment Method 0
Manager 0
City 0
Sales 0
dtype: int64
```

Duplicate Rows:

9

**After Preprocessing:**

(254, 9)

```

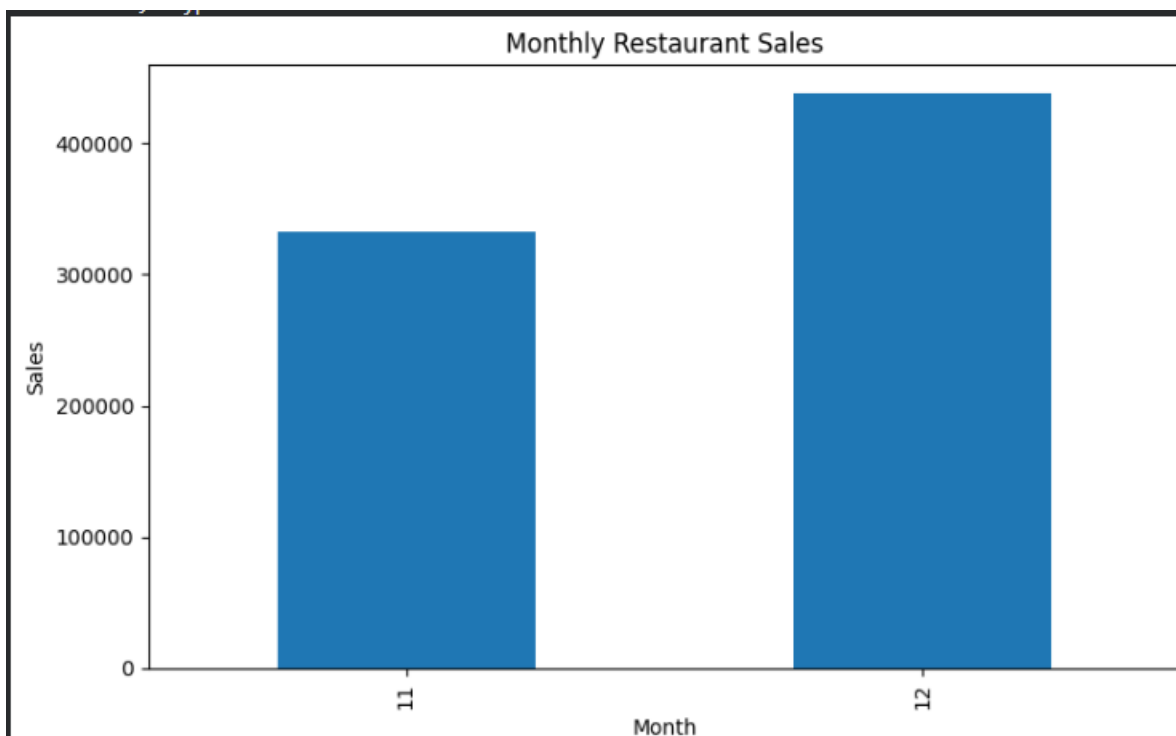
=====
KDD - TRANSFORMATION
=====
Transformed Data:
 Date Product Price Quantity Purchase Type \
0 2022-11-07 Fries 3.49 573.07 Online
1 2022-11-07 Beverages 2.95 745.76 Online
2 2022-11-07 Sides & Other 4.99 200.40 In-store
3 2022-11-08 Burgers 12.99 569.67 In-store
4 2022-11-08 Chicken Sandwiches 9.95 201.01 In-store

 Payment Method Manager City Sales Day Month Year \
0 Gift Card Tom Jackson London 2000.0143 7 11 2022
1 Gift Card Pablo Perez Madrid 2199.9920 7 11 2022
2 Gift Card Joao Silva Lisbon 999.9960 7 11 2022
3 Credit Card Walter Muller Berlin 7400.0133 8 11 2022
4 Credit Card Walter Muller Berlin 2000.0495 8 11 2022

 DayOfWeek
0 0
1 0
2 0
3 1
4 1

Monthly Sales:
Month
11 332114.6584
12 437401.2008
Name: Sales, dtype: float64

```



```

=====
KDD - DATA MINING
=====

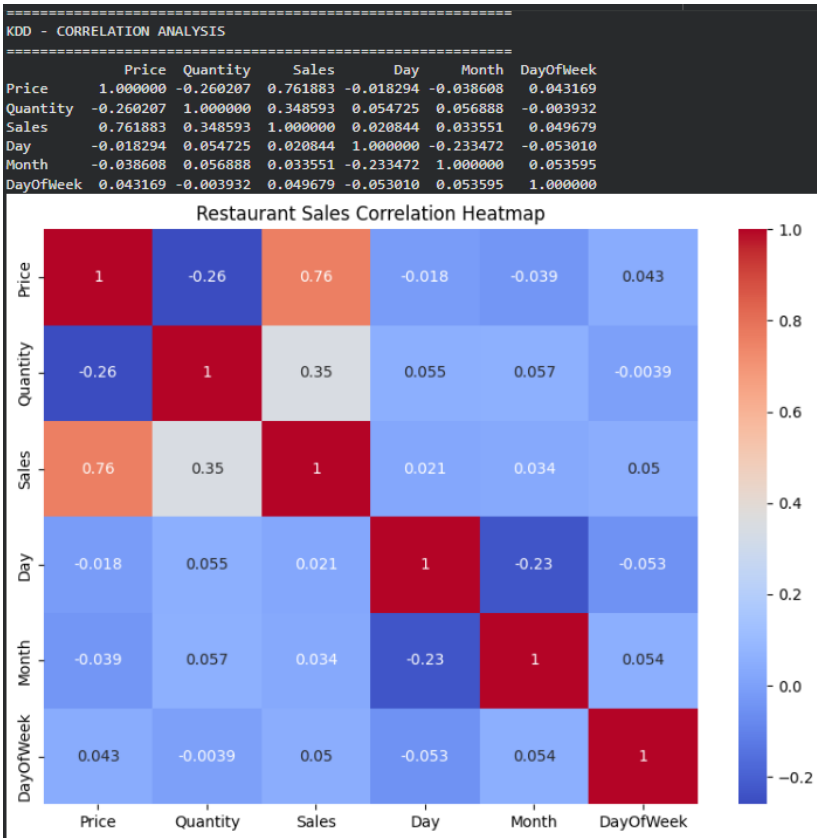
Best-Selling Products:
Product
Burgers 376999.8069
Fries 125674.2903
Chicken Sandwiches 114641.6950
Beverages 103200.2630
Sides & Other 48999.8040
Name: Sales, dtype: float64

Sales by City:
City
Lisbon 241714.1157
London 211201.0406
Madrid 136200.2665
Berlin 100600.1313
Paris 79800.3051
Name: Sales, dtype: float64

Average Sales by Product:
Product
Burgers 7249.996287
Fries 2464.201771
Chicken Sandwiches 2204.647981
Beverages 2064.005260
Sides & Other 999.996000
Name: Sales, dtype: float64

Quantity Sold by Product:
Product
Beverages 34983.14
Fries 32034.34
Burgers 29022.31
Chicken Sandwiches 11135.92
Sides & Other 9819.60
Name: Quantity, dtype: float64

```



```

=====
KDD - INTERPRETATION
=====
Highest Selling Product: Burgers
Highest Sales City: Lisbon
Total Sales: 769515.8592000001
Average Sales: 3029.589996850394
Highest Product Sales: 376999.8069

=====
CRISP-DM - BUSINESS UNDERSTANDING
=====

Business Objective:
Analyze restaurant sales data and predict restaurant sales.

Goals:
1. Understand restaurant sales patterns.
2. Identify best-selling products.
3. Identify high-performing cities.
4. Analyze purchase methods.
5. Analyze payment methods.
6. Predict restaurant sales.
7. Support business and inventory planning.

=====
CRISP-DM - DATA UNDERSTANDING
=====
Number of Rows: 254
Number of Columns: 14

Column Names:
['Order ID', 'Date', 'Product', 'Price', 'Quantity', 'Purchase Type', 'Payment Method', 'Manager', 'City', 'Sales', 'Day', 'Month', 'Year', 'DayOfWeek']

```

```

Data Types:
Order ID int64
Date datetime64[ns]
Product object
Price float64
Quantity float64
Purchase Type object
Payment Method object
Manager object
City object
Sales float64
Day int32
Month int32
Year int32
DayOfWeek int32
dtype: object

Missing Values:
Order ID 0
Date 0
Product 0
Price 0
Quantity 0
Purchase Type 0
Payment Method 0
Manager 0
City 0
Sales 0
Day 0
Month 0
Year 0
DayOfWeek 0
dtype: int64

```

```

Statistical Summary:
 Order ID Date Price Quantity \
count 254.000000 254 254.000000 254.000000
mean 10584.133858 2022-12-03 10:23:37.322834688 7.102323 460.611457
min 10452.000000 2022-11-07 00:00:00 2.950000 200.400000
25% 10520.250000 2022-11-21 00:00:00 3.490000 201.010000
50% 10583.500000 2022-12-03 00:00:00 4.990000 538.880000
75% 10649.750000 2022-12-16 18:00:00 9.950000 677.440000
max 10713.000000 2022-12-29 00:00:00 29.050000 754.430000
std 75.889181 NaN 4.341855 214.888699

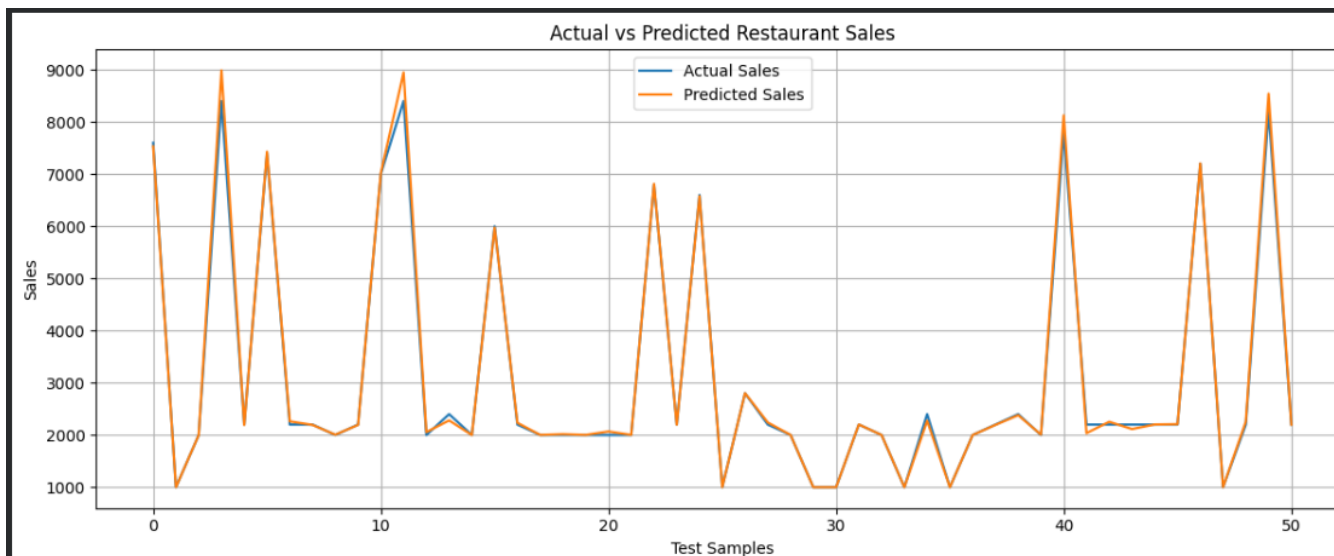
 Sales Day Month Year DayOfWeek
count 254.000000 254.000000 254.000000 254.0 254.000000
mean 3029.589997 16.779528 11.555118 2022.0 2.842520
min 999.996000 1.000000 11.000000 2022.0 0.000000
25% 2000.011500 11.000000 11.000000 2022.0 1.000000
50% 2100.020400 17.000000 12.000000 2022.0 3.000000
75% 2400.003200 23.750000 12.000000 2022.0 4.750000
max 16074.435000 30.000000 12.000000 2022.0 6.000000
std 2420.118378 7.884957 0.497934 0.0 1.955736

=====
CRISP-DM - DATA PREPARATION
=====
Prepared Dataset Shape:
(254, 25)

=====
CRISP-DM - MODELING
=====
Random Forest Regression Model Trained Successfully!

=====
CRISP-DM - EVALUATION
=====
MAE: 57.77373168627352
MSE: 18800.63379886508
RMSE: 137.11540321519345
R2 Score: 0.9964548624210184

```



```

=====
CRISP-DM - DEPLOYMENT / BUSINESS INSIGHTS
=====
Total Restaurant Sales: 769515.8592000001
Top Selling Product: Burgers
Top Performing City: Lisbon
Top Purchase Type: Online

```

```
=====
FINAL PROJECT SUMMARY
=====

Restaurant Sales Analysis Completed Successfully!

Frameworks Applied:
1. SEMMA
2. KDD
3. CRISP-DM

Techniques Applied:
- Data Loading
- Data Cleaning
- EDA
- Statistical Analysis
- Grouping and Aggregation
- Data Visualization
- Correlation Analysis
- Feature Engineering
- One-Hot Encoding
- Machine Learning
- Sales Prediction
- Model Evaluation

Machine Learning Algorithm:
Random Forest Regression

Target:
Sales = Price x Quantity

PROJECT COMPLETED SUCCESSFULLY!
```

## Result

The Restaurant Sales dataset was successfully imported, cleaned and analyzed using Exploratory Data Analysis. Sales was derived from Price and Quantity, and the data was examined using statistical measures, grouping, aggregation, visualization and correlation analysis. The SEMMA, KDD and CRISP-DM frameworks were applied to organize the complete data-analysis workflow. A Random Forest Regression model was trained to predict restaurant sales and evaluated using MAE, MSE, RMSE and  $R^2$  score. The analysis provides useful insights into product performance, city-wise sales and sales trends that can support restaurant business planning.